```
from collections import deque

def ac3(domains, neighbors):
    """
    AC-3 algorithm for constraint propagation.
    It enforces arc consistency between variables (regions).
    """
    queue = deque()
    # Initialize queue with all arcs (variable pairs)
    for var in neighbors:
        for neighbor in neighbors[var]:
            queue.append((var, neighbor))

    while queue:
        xi, xj = queue.popleft()
        if remove_inconsistent_values(xi, xj, domains):
            # If domain of xi was reduced, add arcs (xk, xi) back t
            if len(domains[xi]) == 0:
                return False  # Failure, no legal values left
            for xk in neighbors[xi]:
                if xk != xj:
                    queue.append((xk, xi))
    return True

def remove_inconsistent_values(xi, xj, domains):
    """
    Remove values from xi's domain that have no compatible value in
    """
    removed = False
    for x in set(domains[xi]):  # Use set to avoid modifying during
        # If no value y in xj's domain allows (x != y), remove x fr
        if all(x == y for y in domains[xj]):
            domains[xi].remove(x)
            removed = True
    return removed

def backtrack(assignment, domains, neighbors):
    """
    Backtracking search to assign colors to all regions.
    """
    # If all variables assigned, return assignment
    if len(assignment) == len(domains):
        return assignment

    # Select unassigned variable with smallest domain (MRV heuristi
    unassigned_vars = [v for v in domains if v not in assignment]
    var = min(unassigned_vars, key=lambda v: len(domains[v]))

    for value in domains[var]:
        # Check if value consistent with assignment (no neighbor ha
        if all(assignment.get(neigh) != value for neigh in neighbor
            assignment[var] = value
```

```
            # Make a copy of domains for constraint propagation
            local_domains = {v: list(domains[v]) for v in domains}
            local_domains[var] = [value]

            # Propagate constraints
            if ac3(local_domains, neighbors):
                result = backtrack(assignment, local_domains, neigh
                if result:
                    return result

            # If failure, remove assignment and try next value
            assignment.pop(var)

    return None

# Example: Map coloring for Australia
def main():
    # Variables = regions
    variables = ['WA', 'NT', 'SA', 'Q', 'NSW', 'V', 'T']

    # Domains = possible colors for each region
    colors = ['Red', 'Green', 'Blue']
    domains = {var: colors[:] for var in variables}

    # Neighbors (constraints): adjacent regions must have different
    neighbors = {
        'WA': ['NT', 'SA'],
        'NT': ['WA', 'SA', 'Q'],
        'SA': ['WA', 'NT', 'Q', 'NSW', 'V'],
        'Q': ['NT', 'SA', 'NSW'],
        'NSW': ['Q', 'SA', 'V'],
        'V': ['SA', 'NSW'],
        'T': []  # Tasmania has no neighbors
    }

    assignment = {}
    solution = backtrack(assignment, domains, neighbors)

    if solution:
        print("Solution found:")
        for region in sorted(solution):
            print(f"{region}: {solution[region]}")
    else:
        print("No solution found.")

if __name__ == "__main__":
    main()
```

```
Solution found:
NSW: Green
NT: Green
Q: Red
SA: Blue
T: Red
```

```
V: Red
WA: Red
```

```
import math

board = ['' for _ in range(9)]

def print_board():
    for i in range(3):
        print(board[3*i], '|', board[3*i+1], '|', board[3*i+2])
        if i < 2:
           print('---------')

def is_winner(b, player):

     wins = [
      (0,1,2), (3,4,5), (6,7,8),
      (0,3,6), (1,4,7), (2,5,8),
      (0,4,8), (2,4,6)
     ]
     for a,b_,c in wins:
        if b[a] == b[b_] == b[c] == player:
            return True
     return False

def is_board_full(b):
  return '' not in b

def minimax(b, depth, is_maximizing):

  if is_winner(b, 'X'):
    return 10 - depth
  if is_winner(b, '0'):
    return depth - 10
  if is_board_full(b):
    return 0

  if is_maximizing:
    best_score = -math.inf
    for i in range(9):
      if b[i] == '':
        b[i] = 'X'
        score = minimax(b, depth + 1, False)
        b[i] = ''
        best_score = max(score, best_score)
    return best_score
  else:
    best_score = math.inf
    for i in range(9):
      if b[i] == '':
        b[i] = '0'
        score = minimax(b, depth + 1, True)
        b[i] = ''
        best_score = min(score, best_score)
```

```
    return best_score

def best_move():

  best_score = -math.inf
  move = -1
  for i in range(9):
    if board[i] == '':
      board[i] = 'X'
      score = minimax(board, 0, False)
      board[i] = ''
      if score > best_score:
        best_score = score
        move = i
  return move

def play_game():

  print("Board positions are numbered 0 to 8:")
  print("0 | 1 | 2")
  print('---------')
  print("3 | 4 | 5")
  print('---------')
  print("6 | 7 | 8")
  print()

  while True:

    move = best_move()
    if move == -1:
      print("Game over! It's a tie.")
      break
    board[move] = 'X'
    print("AI (X) plays move:", move)
    print_board()
    if is_winner(board, 'X'):
        print("AI (X) wins!")
        break
    if is_board_full(board):
        print("Game over! It's a tie.")
        break

    while True:
       try:
          human_move = int(input("Enter your move (0-8): "))
          if 0 <= human_move <= 8 and board[human_move] == '':
            board[human_move] = '0'
            break
          else:
               print("Inavalid move. Try again.")
       except ValueError:
        print("Please enter a valid number.")

    print_board()
```

```
        if is_winner(board, '0'):
            print("You win!")
            break
        if is_board_full(board):
            print("Game over! It's a tie.")
            break

if __name__ == "__main__":
  play_game()
```

```
Board positions are numbered 0 to 8:
0 | 1 | 2
---------
3 | 4 | 5
---------
6 | 7 | 8

AI (X) plays move: 0
X |  |
---------
 |  |
---------
 |  |
Enter your move (0-8): 2
X |  | 0
---------
 |  |
---------
 |  |
AI (X) plays move: 3
X |  | 0
---------
X |  |
---------
 |  |
Enter your move (0-8): 52
Inavalid move. Try again.
Enter your move (0-8): 8
X |  | 0
---------
X |  |
---------
 |  | 0
AI (X) plays move: 6
X |  | 0
---------
X |  |
---------
X |  | 0
AI (X) wins!
```

```
import numpy as np
from queue import PriorityQueue

def a_star_search(graph, start, goal, h):
  open_set = PriorityQueue()
  open_set.put((0, start))
```

```
  g_score = {node : float('inf') for node in graph}
  g_score[start] = 0

  came_from = {}

  while not open_set.empty():
    current_f, current = open_set.get()

    if current == goal:
      path = []
      while current in came_from:
        path.append(current)
        current = came_from[current]
      path.append(start)
      path.reverse()
      return path, g_score[goal]

    for neighbhor, cost in graph[current]:
      new_cost = g_score[current] + cost

      if new_cost < g_score[neighbhor]:
        g_score[neighbhor] = new_cost
        priority = new_cost + h(neighbhor)
        open_set.put((priority, neighbhor))
        came_from[neighbhor] = current

  return None, float('inf')

graph = {
    'A': [('B', 1),('C', 4)],
    'B': [('A', 1),('C', 2), ('D', 5)],
    'C': [('A', 4),('B', 2), ('D', 1)],
    'D': [('B', 5), ('C', 1)]
}

heuristic = {
    'A': 7,
    'B': 6,
    'C': 2,
    'D': 0
}

def h(node):
  return heuristic[node]

path, cost = a_star_search(graph, 'A', 'D', h)

print("Path:", path)
print("Cost:", cost)
```

```
Path: ['A', 'C', 'D']
Cost: 5
```